Deep Reinforcement Learning for Disturbance-Robust Ball-on-Tray Balancing

ME5418 – Machine Learning in Robotics | Stage 3: Learning Agent Code & Report

This stage turns the stage 2 networks into learning agents: complete training loops for both algorithms of the proposal, **PPO** and **SAC**, the proposal's **staged domain randomisation**, and the tools to evaluate a trained agent against the PD and LQR baselines and watch it in the 3D view. Everything is written against `torch.nn`, with no RL framework, and runs on one CPU core. The code is complete and runnable, and the reference runs (three training seeds each) learn: a 1M-step PPO run takes 6–7 minutes and ends level with LQR on the 100 test seeds (−3.7 on average); under noise, lag and delay, PPO trained in those conditions is also level with LQR and below PD. SAC learns faster per sample but levels off below PD.

1. Code Explanation

Stages 1–2 are unchanged apart from additions to `networks/losses.py` and a `--checkpoint` option in `demo.py`. Stage 3 adds the `agents/` package and three scripts.

File	Content
`agents/common.py`	Curriculum, parallel environments (`VecEnv`), evaluation on fixed seeds, CSV logging.
`agents/ppo.py`	PPO loop: rollouts, GAE, minibatch epochs, learning-rate annealing; optional asymmetric critic.
`agents/sac.py`	SAC loop: replay buffer, target critic (Polyak), automatic entropy tuning, random warm-up.
`networks/losses.py`	+ `gae` (GAE(λ) with bootstrapping at truncation), + `sac_alpha_loss`, + `use_min` option of the SAC actor loss.
`train.py`	Command line: `--algo ppo|sac`, `--curriculum`, `--realistic`, `--asymmetric`, `--quick`; writes `runs/<name>/`.
`evaluate_agent.py`	Agents next to PD and LQR on the test seeds 0–99, with paired differences; `--realistic`.
`plot_training.py`	Learning curves with the PD and LQR levels (Fig. 1).
`tests/test_agents.py`	20 tests: GAE by hand, `VecEnv`, curriculum, replay buffer, Polyak, α direction, smoke runs, reproducibility.

1.1 Pipeline and shared pieces

`train.py` builds a configuration (saved as `config.json`) and calls `train_ppo` or `train_sac`. Both loops draw experience from `VecEnv`, update the stage 2 networks with the stage 2 losses, append a row to `progress.csv`, and every 50k (PPO) / 10k (SAC) steps evaluate the deterministic policy on seeds 500–519, saving `best.pt` and `last.pt`. **Curriculum.** The proposal's "fixed friction and no disturbances first, then progressively randomise" is four stages: fixed $c_{rr} = 0.02$, no pulses (10 % of steps) → $c_{rr} \in [0.005, 0.05]$ (15 %) → + pulses of 0.5–1.25 m s^{-2} (15 %) → full range, pulses up to 2.0 m s^{-2} (60 %); `--realistic` adds noise, lag and delay to the last stage. A stage changes at each environment's next episode, never inside one. **VecEnv** steps N environments and batches the network call. We wrote it instead of using `gym.vector` because both loops need the *true* last observation of an episode (and its `info`, for the asymmetric critic) separately from the observation after the automatic reset, and every episode needs its own seed for exact reproducibility. **Seeds.** Training episodes use seeds ≥ 100,000; model selection uses 500–519; the final comparison uses 0–99, the seeds of the stage 1 baseline table, seen by nothing else.

1.2 The two training loops

	PPO (`agents/ppo.py`)	SAC (`agents/sac.py`)
Data	on-policy: 8 envs × 256 steps = 2,048 per iteration, used once	off-policy: 1 env, every transition kept in a 300k replay buffer; 5,000 random warm-up actions
Update	10 epochs × minibatches of 256; clipped loss (ε = 0.2), value weight 0.5	1 update per step, batch 256: twin-Q critic, then actor, then α, then target
Value target	GAE, $\gamma = 0.99$, $\lambda = 0.95$	$r + \gamma(1-d)[\min Q'(s', a') - \alpha \log \pi(a'\|s')]$ from the **target** critic, $\tau = 0.005$
Exploration	free log σ, no entropy bonus (σ decays 0.61 → 0.10)	α learned so that the entropy tracks −2 (= −action dim.)
Stabilisers	advantages normalised per minibatch; grad-norm clip 0.5; lr $3 \cdot 10^{-4} \to 0$ linearly	actor maximises min(Q1, Q2); critic frozen during the actor's backward pass

Both scale the reward by 0.1 for learning (value targets near 10 instead of 150; logs are unscaled), and both treat the two episode ends differently: a fall (`terminated`) is terminal, the 500-step limit (`truncated`) is not, so

$$\delta_t = r_t + \gamma\,(1 - term_t)\,V(s_{t+1}) - V(s_t), \quad A_t = \delta_t + \gamma\lambda\,(1 - done_t)\,A_{t+1}$$

with s_{t+1} the *true* last observation, not the next episode's first one that an auto-resetting environment returns. A balanced episode always ends by truncation, so treating it as terminal would teach the critic that balancing longer is worth nothing; a hand-computed GAE test pins down both cases.

1.3 First results

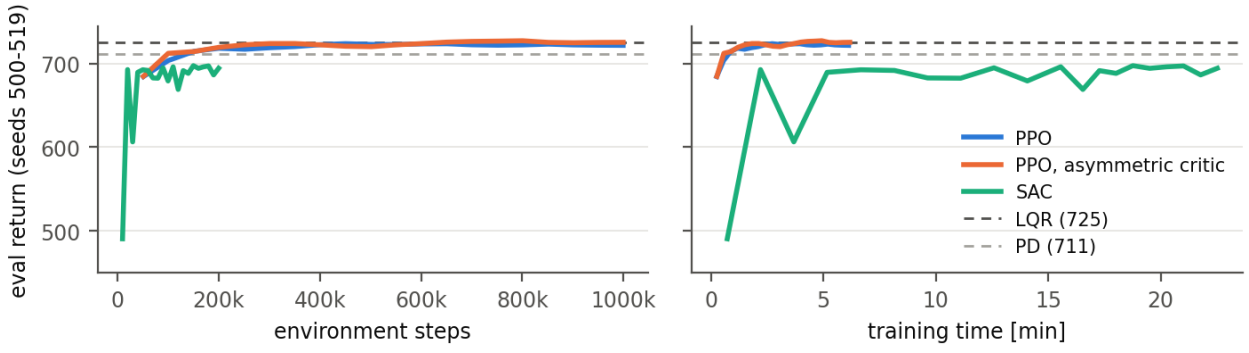


Fig. 1. Evaluation return (deterministic policy, seeds 500–519, full range) of the reference runs, against environment steps and training time; dashed: LQR and PD on the same seeds. Training seed 0 shown; twelve runs at once, one CPU core each: PPO 1M steps in 6.2–6.6 min, SAC 200k steps in 22.5 min.

Condition	Policy	Return, best.pt	Survival	Goal %	vs LQR, paired; seeds better	last.pt
Ideal	PD	712.6 ± 11.7	100 %	93.4	—	
	LQR	724.4 ± 8.3	100 %	96.1	—	
	PPO	720.8 (715.9–726.5)	99–100 %	95.0	−3.7 (−8.5…+2.1); 57–80	720.0
	PPO, asymmetric critic	721.7 (719.1–725.4)	99–100 %	95.2	−2.7 (−5.3…+1.0); 74–79	721.0
	SAC	685.3 (672.8–694.5)	99–100 %	88.1	−39.1 (−51.6…−29.9); 0–1	679.8
Noise, lag 0.05 s, 1-step delay	PD	658.5 ± 68.3	99 %	86.7	—	
	LQR	627.4 ± 111.1	97 %	78.8	—	
	PPO trained on the ideal task	358.4 (236.2–454.9)	32–83 %	17.6	−269.0 (−391.2…−172.5); 0–1	
	PPO trained with `--realistic`	625.3 (618.9–628.8)	95–97 %	78.2	−2.1 (−8.6…+1.4); 34–47	625.7

Table 1. `evaluate_agent.py` on the test seeds 0–99 (full randomisation range; max. return 750). Agents: mean over training seeds 0–2 (range in brackets); `best.pt` is the best checkpoint of the last curriculum stage on seeds 500–519, `last.pt` the end of the run. "Paired" compares the same test seed (same start, c_{rr}, pulses); "seeds better" counts the test seeds on which the agent beats LQR, range over training seeds.

2. Existing Codes / Libraries

What we used. PyTorch 1.13.1 remains the only ML dependency: `optim.Adam`, `nn.utils.clip_grad_norm_`, autograd, `torch.save`. gym 0.26.2 provides only the environment API; numpy runs the replay buffer and the episode bookkeeping; matplotlib draws Fig. 1. **What we wrote.** Both training loops, GAE, the replay buffer, the target update, entropy tuning, the curriculum, the vectorised environment and the evaluation are ours. As in stage 2 we did not use stable-baselines3 or RLlib: they would hide exactly what this stage is about, current stable-baselines3 requires `gymnasium`, and its PPO gives actor and critic the same input, so the asymmetric critic would need a custom policy class anyway. **What we borrowed.** Hyper-parameters, not code: the PPO values are close to the MuJoCo settings of Schulman et al. (2017) and the stable-baselines3 defaults; the SAC values and the target entropy −dim(A) follow Haarnoja et al. (2018b). **What we had to do to make it work.** `torch.set_num_threads(1)` (fastest for 22k-parameter networks and bit-reproducible: two runs with one seed give identical weights, which a test checks); every checkpoint stores its own configuration so that `evaluate_agent.py` and `demo.py` can rebuild any run; the code stays Python 3.7-compatible.

3. Reflections and Lessons Learned

3.1 Changes from the proposal and from stage 2

- **The curriculum became concrete.** The proposal said "progressively"; we had to choose stages and fractions (§1.1). The first stage turned out to be sufficient for a surprisingly good policy: PPO evaluated on the *full* task after 51k steps of only fixed- friction, pulse-free training already scores 683–690 with 100 % survival on 20 evaluation seeds (three training seeds): it generalises to frictions and pushes it has not seen yet.
- **SAC's actor now uses min(Q1, Q2)** instead of stage 2's Q1, and α is learned instead of fixed at 0.2 (stage 2 deferred both). In all three runs α fell from 0.2 to below 0.003 within 28k–32k steps while the policy entropy settled at the target −2.
- **No PPO entropy bonus** (stage 2 default: 0.01). With a state-independent log σ, exploration is set by σ itself, which PPO lowers as the policy improves (0.61 → 0.10–0.11 over each run); we have not yet tested whether a bonus helps.
- **Reward scaling** (×0.1, learning only, not in the proposal) keeps value targets near 10; chosen by reasoning, not by ablation.

3.2 Lessons

- **Mean return hides the tail.** On the test seeds PPO beats LQR on 57–80 of 100 seeds depending on the training seed, yet its mean is level with or below LQR's, because four of the six PPO models drop the ball once: seed 0 starts near a corner with a 1.41 m s^{-2} push from t = 0.03 s. LQR saves it. Single-run win counts are unreliable: with the same configuration they ranged from 34 to 80. The proposal's success-rate metric is the right primary metric; the final version needs targeted hard-start tests.
- **Training under noise, lag and delay does not yet beat the baselines.** PPO trained on the ideal task collapses under them (358.4, 32–83 % survival); trained to the end of the `--realistic` curriculum it is level with LQR (−8.6 to +1.4) and below PD in every run (29.7–39.6 lower, paired), while an early checkpoint trained only on the ideal fixed-friction task (step 51,200) is level with PD (657.3, paired −1.2; not yet explained). The proposal expected RL to be more robust than a PID-type controller (here PD) under realistic conditions; so far it only matches LQR and stays below PD, which does not meet that expectation.
- **Sample efficiency ≠ speed.** SAC scored 680–693 after 20k steps in two of three runs (551 in the third), where PPO had 683–690 after 51k, but each SAC step costs a gradient update: its 200k steps took 22.5 min, PPO's 1M took 6.2–6.6. SAC then levelled off below PD in all three runs (test return 672.8–694.5). We do not know why yet; candidates are the target entropy, the reward scale, and simply too few steps.

3.3 Remaining changes towards the final version

More training seeds per configuration with confidence intervals (three so far); the proposal's two remaining baselines (LQR at nominal friction, and the grid-searched fixed gain); held-out conditions including 3 m s^{-2} pulses; the asymmetric critic and `--curriculum full` as ablations (with three seeds the two critics end level, 721.7 vs 720.8 on the test seeds, so stage 2's open question is still open); hard-start episodes to close the tail failure; a diagnosis of SAC's plateau; and a realistic-condition policy that beats PD.

4. How to Run

```
python train.py --algo ppo|sac [--realistic] [--asymmetric] [--seed N]  # PPO 1M steps ~6–7 min, SAC 200k ~23 min
python evaluate_agent.py trained/ppo_staged_seed0.pt [--realistic]  # a reference agent vs PD, LQR (Table 1)
```

All 68 tests (14 environment, 7 renderer, 8 realism, 19 network, 20 agent) pass; `README.md` documents every option.